UE5_Spout2_DX12

Technical Changes vs. the Original Repository

Internal engineering note

2026-07-07

Abstract

This document describes, at a technical level, every modification applied to the `Spout2_DX12` plugin on top of the base version tracked in `origin/main` (commit `37e714e`, "Readme update and hot fix for viewport sender" lineage). The changes were developed and validated in a separate working copy of the plugin embedded in an Unreal Engine 5.8 project, then ported back into this repository. They fall into four categories: a GPU-hang fix for headless (`-RenderOffScreen`) sessions, a Shipping-build shutdown crash fix, a receiver-side frame-synchronization fix, and a linker fix for the third-party Spout libraries. A command-line build script was also added.

Contents

1 Scope and Baseline **1**

2 Fix 1 — Deferred Bridge Start (Headless GPU Hang) **1**
2.1 Symptom . . . 1
2.2 Root Cause . . . 1
2.3 Fix Design . . . 2
2.4 Lifecycle Cleanup . . . 3
2.5 Files and Symbols . . . 3

3 Fix 2 — Shipping Shutdown Crash **3**
3.1 Symptom . . . 3
3.2 Root Cause . . . 3
3.3 Fix . . . 3

4 Fix 3 — Graceful Stop Channel for Headless Sessions **3**
4.1 Problem . . . 3
4.2 Fix: File-Based Command Channel . . . 3

5 Fix 4 — Receiver Frame Synchronization **4**
5.1 Problem . . . 4
5.2 Fix . . . 4

6 Fix 5 — Third-Party Linker Configuration **5**
6.1 Symptom . . . 5
6.2 Root Cause . . . 5
6.3 Fix . . . 5

7 Addition — Command-Line Build Script **6**

8 Summary Table **6**

1 Scope and Baseline

- **Baseline**: commit `37e714e` (source-identical to `baf099f`, which only touches `README.md`).
- **Files touched**: 7 source files, 476 insertions / 9 deletions.
  - `Source/Spout2_DX12/Private/Spout2_DX12.cpp`
  - `Source/Spout2_DX12/Private/SpoutReceiverComponent.cpp`
  - `Source/Spout2_DX12/Private/SpoutSenderComponent.cpp`
  - `Source/Spout2_DX12/Public/Spout2_DX12.h`
  - `Source/Spout2_DX12/Public/SpoutReceiverComponent.h`
  - `Source/Spout2_DX12/Public/SpoutSenderComponent.h`
  - `Source/Spout2_DX12/Spout2_DX12.Build.cs`
- **Binaries**: not carried over as-is; recompiled against this repository's copy of the source via the new `Build.bat` (Section 7), verified to produce a successful Editor / Game Development / Game Shipping build for Win64 against UE 5.8.

2 Fix 1 — Deferred Bridge Start (Headless GPU Hang)

2.1 Symptom

In windowless sessions launched with `-RenderOffScreen`, the process intermittently hung with `DXGI_ERROR_DEVICE_HUNG` on the D3D12 graphics queue, with GPU crash-dump breadcrumbs consistently stuck at *Frame 0*. The failure rate observed during validation was approximately 3 out of 4 runs before the fix, 0 out of 4 after.

2.2 Root Cause

Both `USpoutSenderComponent` and `USpoutReceiverComponent` open a `D3D11On12` interop bridge (via `spoutDX12`) to share textures with other Spout-aware processes. This bridge submits interop work on the same D3D12 queue the engine uses for rendering. In a normal windowed session, the swapchain's `Present` call implicitly paces that queue every frame. In a windowless session there is no swapchain, so nothing paces the queue during the engine's startup phase — which is also when the Pipeline State Object (PSO) precompilation flood happens (map load, first-frame shader compiles). Opening the interop bridge and submitting work while that flood is in flight was racing the queue and deadlocking it.

2.3 Fix Design

The actual bridge initialization in both components is now held behind two conditions, checked at the top of `StartBroadcastConfigured()` (sender) and `StartReceiving()` (receiver):

1. **Minimum grace period**: world time must have reached `StartDelaySeconds` (a new `UPROPERTY`, default `2.0f`; `0.0f` disables the whole mechanism and restores pre-fix immediate-start behavior).
2. **PSO precaching idle**: both `PipelineStateCache::NumActivePrecacheRequests()` and `FShaderPipelineCache::NumPrecompilesRemaining()` must be zero.

A hard cap, `MaxStartDelaySeconds` (new `UPROPERTY`, default `15.0f`; `0.0f` = no cap), forces the start even if PSOs are still compiling once world time exceeds it — this is a safety valve so a scene with a pathologically long shader-compile tail cannot hold the sender/receiver back indefinitely.

While deferred, a one-shot `FTimerHandle` (`StartDelayTimerHandle`) re-invokes the same entry point after a retry delay: the remaining grace-period time if still inside it, otherwise a fixed `0.5 s`

poll interval. The mechanism only engages in an actual game world (`World->IsGameWorld()`); the editor/PIE path is unaffected because a window's `Present` already paces the queue there.

Listing 1: Deferred-start gate (sender), abridged

```
UWorld* World = GetWorld();
if (World && World->IsGameWorld() && StartDelaySeconds > 0.0f)
{
    const float WorldSeconds = static_cast<float>(World->GetTimeSeconds
        ());
    const bool bMinTimePassed = WorldSeconds >= StartDelaySeconds;

    const int32 PendingPSOs =
        static_cast<int32>(PipelineStateCache::NumActivePrecacheRequests
            ()) +
        static_cast<int32>(FShaderPipelineCache::NumPrecompilesRemaining
            ());

    const bool bForcedStart =
        MaxStartDelaySeconds > 0.0f && WorldSeconds >=
            MaxStartDelaySeconds;

    if (!bForcedStart && (!bMinTimePassed || PendingPSOs > 0))
    {
        // stash config, arm a retry timer, bail out without
        // opening the D3D11On12 bridge
        return;
    }
    // otherwise: proceed with the real bridge initialization
}
```

2.4 Lifecycle Cleanup

`StopBroadcastInternal()` / `StopReceivingInternal()` now explicitly clear `StartDelayTimerHandle` via `World->GetTimerManager().ClearTimer(...)`. Without this, stopping the sender/receiver while a start was deferred would leave an armed timer that silently restarted the broadcast/receive path up to `0.5 s` later, independent of the explicit stop. A companion flag, `bLoggedStartDeferral`, throttles the "holding start" log line to once per deferral cycle instead of once per `0.5 s` retry, and is reset on stop so a future deferral logs again.

2.5 Files and Symbols

File	New symbols
`SpoutSenderComponent.h`	`StartDelaySeconds`, `MaxStartDelaySeconds`, `StartDelayTimerHandle`, `bLog`
`SpoutSenderComponent.cpp`	gate in `StartBroadcastConfigured()`; cleanup in `StopBroadcastInternal(`
`SpoutReceiverComponent.h`	same four symbols (independent per-component state)
`SpoutReceiverComponent.cpp`	gate in `StartReceiving()`; cleanup in `StopReceivingInternal()`

3 Fix 2 — Shipping Shutdown Crash

3.1 Symptom

Access violation observed during process teardown in monolithic Shipping builds.

3.2 Root Cause

`FSpout2_DX12Module::ShutdownModule()` used to call `FPlatformProcess::FreeDllHandle(MyDllHandle)` to explicitly unload `SpoutDX12.dll`. In monolithic Shipping builds, static destructors running *after* module shutdown can still call into that DLL; once it has been unloaded, that call lands on unmapped memory.

3.3 Fix

The explicit `FreeDllHandle` call was removed; `MyDllHandle` is simply reset to `nullptr` as a bookkeeping value. The DLL is left mapped for the remaining lifetime of the process — harmless, since the OS reclaims all module handles unconditionally at process exit. This trades an unnecessary explicit unload for guaranteed safety against late static-destructor calls.

4 Fix 3 — Graceful Stop Channel for Headless Sessions

4.1 Problem

A windowless process has no `WM_CLOSE` to receive, so there is no clean way to ask it to exit from the outside. Terminating it with `taskkill /F` skips Unreal's orderly shutdown path (`EndPlay` → `StopBroadcastInternal` → Spout sender release), leaving a "ghost" sender registered in Spout's shared memory. The next launch's sender then gets auto-renamed (`"Sender Component_1"`, `"_2"`, ...) by Spout's naming collision handling, breaking any receiver (LiveBoard, OBS, ...) hard-wired to the original fixed name.

4.2 Fix: File-Based Command Channel

`StartupModule()` now computes a per-project stop-file path:

```
StopFilePath = FPaths::Combine(
    FString(FPlatformProcess::UserTempDir()),
    FString::Printf(TEXT("%s_SpoutStop"), FApp::GetProjectName()));
```

i.e. `%TEMP%\<ProjectName>_SpoutStop`. Any stale file from a previous run is deleted immediately on startup (otherwise a new session would see it and self-terminate roughly one second in). A `FTSTicker` callback (`CheckStopFile`), registered on the engine's core ticker at a 1-second period, polls for that file's existence. The core ticker runs even with no viewport, so this works in headless mode.

When the file is detected:

1. It is deleted immediately (one-shot semantics — otherwise it would also kill the *next* session right after startup).
2. `RequestEngineExit(...)` is called — Unreal's *orderly* exit path, as opposed to a hard kill: it lets the current frame finish, tears the world down, and lets `EndPlay`/`StopBroadcastInternal` release the Spout sender from shared memory before the process actually exits (~1.5 s in practice).

Two guards prevent unwanted behavior:

- `GIsEditor` — the channel is inert inside the editor. A file dropped in `%TEMP%` must never be able to close an open Editor session; the channel is only meaningful for packaged/game sessions.
- `IsEngineExitRequested()` — avoids requesting exit twice if it is already in progress.

The ticker handle (`StopFileTickerHandle`) is explicitly removed in `ShutdownModule()` *before* the module object is destroyed, since the delegate was bound with `CreateRaw` (no lifetime tracking) — a tick firing after that point would call into freed memory.

An external tool (e.g. a `.bat` script, not part of this repository) can request a clean shutdown of a running headless instance simply by doing:

```
type nul > "%TEMP%\<ProjectName>_SpoutStop"
```

5 Fix 4 — Receiver Frame Synchronization

5.1 Problem

The receiver copied the sender's shared D3D11 texture on every tick with no synchronization against the sender's write cadence, risking tearing or reading a texture mid-write when sender and receiver frame rates diverge.

5.2 Fix

`USpoutReceiverComponent` now tracks a small piece of Spout frame-sync state:

```
bool bSyncInitialized = false;
char SpoutSyncName[256] = {};
```

In `ReceiveOnce()`, once the active sender's name is known, the mutex/frame-count objects are (re)initialized whenever the sender identity changes (first connection, or a different sender name than last time):

```
if (!bSyncInitialized || FCStringAnsi::Strcmp(SpoutSyncName, SenderName)
    != 0)
{
    if (bSyncInitialized)
    {
        SpoutDX12->frame.CloseAccessMutex();
        SpoutDX12->frame.DisableFrameCount();
    }
    SpoutDX12->frame.CreateAccessMutex(SenderName);
    SpoutDX12->frame.EnableFrameCount(SenderName);
    FCStringAnsi::Strncpy(SpoutSyncName, SenderName, 256);
    bSyncInitialized = true;
}
```

Two checks then guard the actual texture copy:

- **New-frame check**: `SpoutDX12->frame.GetNewFrame()` is a non-blocking read of the sender's frame semaphore; if the sender has not produced a new frame since the last read, the receiver skips the copy this tick. The very first frame (`bConnected == false`) always bypasses this check so the initial image is never dropped.
- **Access mutex**: `SpoutDX12->frame.CheckAccess()` is acquired (non-blocking) immediately before `Ctx11->CopyResource(...)`, and released with `AllowAccess()` immediately after. If the sender is mid-write, `CheckAccess()` returns `false`, the copy is skipped for that tick, and a missed-frame counter (`StatMissedFramesAccum`) is incremented for diagnostics.

Sync state is torn down symmetrically in both `StopReceivingInternal()` and `ReleaseSpoutDevices()` (`CloseAccessMutex()` / `DisableFrameCount()`, then `bSyncInitialized = false` and `SpoutSyncName[0]`

= '\0'), so a subsequent `StartReceiving()` reinitializes cleanly rather than inheriting stale mutex/semaphore handles.

6 Fix 5 — Third-Party Linker Configuration

6.1 Symptom

Crash at runtime tied to `spoutFrameCount` symbol resolution.

6.2 Root Cause

`SpoutDX12.dll` statically embeds the `spoutDX` / `spoutFrameCount` implementation. The module's `Spout2_DX12.Build.cs` previously linked *both* `Spout.lib` and `SpoutDX12.lib`, and delay-loaded *both* `Spout.dll` and `SpoutDX12.dll`. Linking `Spout.lib` caused the linker to resolve `spoutFrameCount` symbols against `Spout.dll` instead of the statically-embedded copy inside `SpoutDX12.dll`. Since `Spout.dll` is never actually loaded at runtime by `SpoutDX12.dll`, any call through those symbols crashed.

6.3 Fix

```
- PublicAdditionalLibraries.Add(Path.Combine(LibDir, "Spout.lib"));
  PublicAdditionalLibraries.Add(Path.Combine(LibDir, "SpoutDX12.lib"));

- PublicDelayLoadDLLs.AddRange(new[] { "Spout.dll", "SpoutDX12.dll" });
+ PublicDelayLoadDLLs.Add("SpoutDX12.dll");
```

Only `SpoutDX12.lib` is linked, and only `SpoutDX12.dll` is delay-loaded; `Spout.dll` is still staged next to the packaged executable as a runtime dependency (unrelated third-party consumers may still need the standalone Spout runtime), but the plugin's own code no longer resolves symbols against it.

7 Addition — Command-Line Build Script

A new `Build.bat` wraps `RunUAT.bat BuildPlugin`, the official Epic tooling for compiling a standalone plugin without an open Editor session (this is the same mechanism used to produce Marketplace plugin binaries). It takes two optional positional arguments, `EngineDir` (default `C:\Program Files\Epic Games\UE_5.8`) and `OutputDir` (default `.\Build\Spout2_DX12`), and invokes:

```
RunUAT.bat BuildPlugin -Plugin=<uplugin path> -Package=<OutputDir> \
    -TargetPlatforms=Win64 -Rocket
```

This compiles three Win64 configurations — Editor, Game Development, Game Shipping — and stages a self-contained, redistributable plugin folder (`.uplugin`, `Binaries/`, `Source/`, `Resources/`) at `OutputDir`. The script was validated end-to-end against UE 5.8: all three configurations compiled and linked successfully (`BUILD SUCCESSFUL`), total wall time $\approx$ 2 minutes on the reference machine. Full usage and installation instructions (per-project vs. engine-wide) are documented separately in `BUILD_INSTRUCTIONS.md`.

8 Summary Table

Change	Failure mode addressed	Mechanism
Deferred bridge start	GPU hang in `-RenderOffScreen` sessions	world-time + PSO-idle gate, capped retry
No explicit DLL unload	Shipping shutdown access violation	leave `SpoutDX12.dll` mapped until process exit
File-based stop channel	Ghost sender after `taskkill /F`	polled stop-file → `RequestEngineExit`
Receiver frame sync	Tearing / mid-write reads	access mutex + frame-count semaphore around `CopyResource`
Linker fix	Crash resolving `spoutFrameCount`	drop `Spout.lib` / `Spout.dll` delay-load
`Build.bat`	No command-line build path	wraps `RunUAT.bat BuildPlugin`